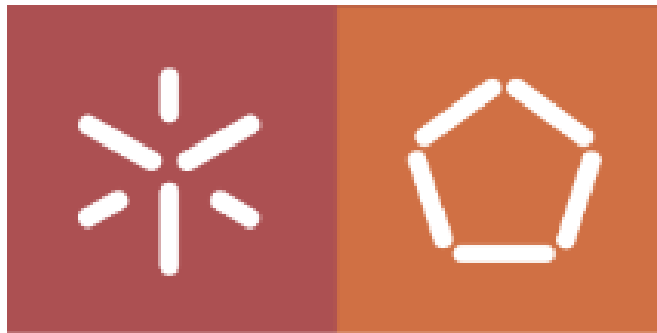


Computação Gráfica

Fase 2 - Transformações Geométricas

Março 2026



Universidade do Minho
Escola de Engenharia

Grupo: 8

Realizado por: Juliana Sofia Vaz da Silva, a105572

Sofia Beatriz Miranda Couto, a106925

Soraia Filipa Ribeiro Pereira, a106806

Índice

Introdução	2
1. Engine	3
1.1. Leitura do ficheiro de configuração	3
1.1.1. Classe Group	3
1.2. Transformações	3
1.2.1. Translação	3
1.2.2. Rotação	4
1.2.3. Escala	5
1.3. Desenho dos Grupos	6
2. Testes	7
3. Sistema Solar	7
4. Extras	9
4.1. Controlo da Câmara	9
Conclusão	11

Introdução

Este relatório descreve o desenvolvimento da segunda fase do projeto da unidade curricular de Computação Gráfica, inserida no 2º semestre do 3º ano da Licenciatura em Engenharia Informática, no ano letivo 2025/2026.

Após a implementação das primitivas geométricas e do motor de visualização básico na fase anterior, esta etapa foca-se na introdução de transformações geométricas e na gestão de cenas hierárquicas.

1. Engine

1.1. Leitura do ficheiro de configuração

1.1.1. Classe *Group*

A classe *Group* é responsável por guardar um vetor de vetores de objetos do tipo *Point* (*models*), uma matriz de transformações (*transformations*) e os subgrupos (*subgroups*):

```
class Group {  
    std::vector<std::vector<Point>> models = {};  
    std::array<std::array<float, 4>, 4> transformations = {  
        {{1, 0, 0, 0},  
         {0, 1, 0, 0},  
         {0, 0, 1, 0},  
         {0, 0, 0, 1}}};  
    std::vector<Group> subgroups = {};  
};
```

1.2. Transformações

1.2.1. Translação

A translação é uma transformação geométrica que move um ponto, ou um conjunto de pontos, numa dada direção e comprimento. Graficamente, os pontos são transladados por aplicação de um vetor $\vec{t}$, que tem a direção e o comprimento da translação. Mais precisamente no contexto tridimensional, a translação de uma translação de um ponto $P = (p_x, p_y, p_z, 1)$ por um vetor (t_x, t_y, t_z) pode ser representada por:

$$P' = P + \vec{t} = (p_x + t_x, p_y + t_y, p_z + t_z)$$

A forma geral de uma matriz de translação por um vetor $\vec{t} (t_x, t_y, t_z)$ em três dimensões é a seguinte:

$$\mathbf{M} = \begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Figura 1 - Matriz de translação associada a um vetor $\vec{t} = (t_x, t_y, t_z)$.

No desenvolvimento deste motor gráfico, optou-se por pré-calcular estas matrizes no momento da leitura do XML para maximizar o desempenho. A matriz de escala acima (Figura 1) é instanciada como um array de 16 posições no formato *column-major* (Figura 2).

De seguida, esta é multiplicada pela matriz atual do grupo, garantindo que as transformações e as proporções são devidamente acumuladas e herdadas por eventuais subgrupos.

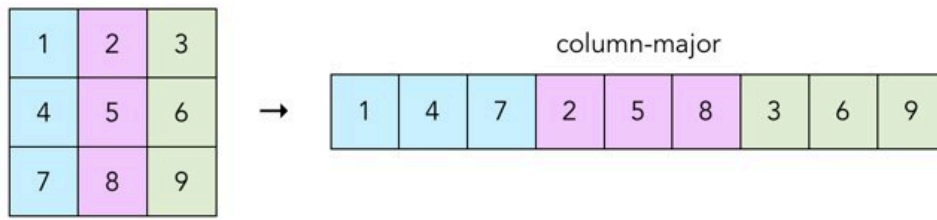


Figura 2 - Exemplo de Column-major order.

Ao aplicar um vetor de translação à matriz base, os objetos representados sofrem um deslocamento espacial correspondente às coordenadas deste vetor. Como se observa na Figura 3, foi aplicada uma translação de 2 unidades no eixo X, -1 no eixo Y e 0 no eixo Z, seguindo o vetor $(2, -1, 0)$.

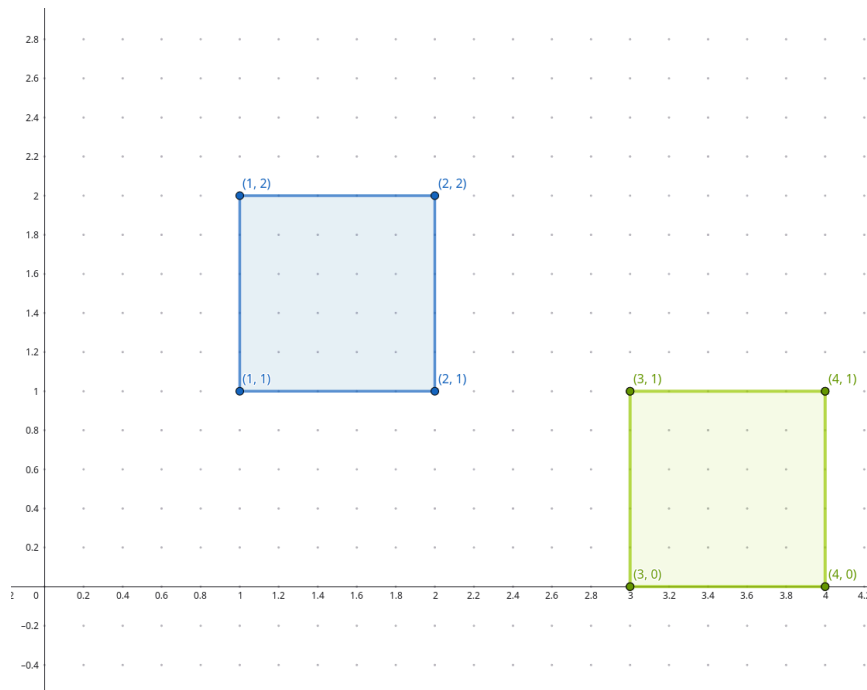


Figura 3 - Exemplo de uma translação segundo o vetor $(2, -1, 0)$.

1.2.2. Rotação

A rotação é uma transformação geométrica que permite reorientar objetos em torno de um eixo no espaço tridimensional.

Matematicamente, a rotação em torno de um eixo arbitrário $v = (x, y, z)$ por um ângulo θ exige que o eixo seja primeiro normalizado para garantir a preservação das proporções do modelo. Utilizando coordenadas homogêneas, a matriz de rotação $R_{v,\theta}$ é definida por:

$$\begin{bmatrix} (1 - \cos \theta) \cdot x^2 + \cos \theta & (1 - \cos \theta) \cdot xy + z \cdot \sin \theta & (1 - \cos \theta) \cdot xz - y \cdot \sin \theta & 0 \\ (1 - \cos \theta) \cdot xy - z \cdot \sin \theta & (1 - \cos \theta) \cdot y^2 + \cos \theta & (1 - \cos \theta) \cdot yz + x \cdot \sin \theta & 0 \\ (1 - \cos \theta) \cdot xz + y \cdot \sin \theta & (1 - \cos \theta) \cdot yz - x \cdot \sin \theta & (1 - \cos \theta) \cdot z^2 + \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Figura 4 - Matriz de rotação em torno de um eixo $v=(x,y,z)$ por um ângulo θ .

Tal como na translação e escala, a matriz é instanciada seguindo o formato *column-major*. Esta matriz resultante é então multiplicada pela matriz de transformações atual do grupo, permitindo que a rotação seja acumulada e herdada corretamente por todos os modelos e subgrupos dependentes.

Após se aplicar a rotação à matriz base, verificamos que os objetos desenhados são rodados no espaço segundo o vetor e ângulo definidos. No exemplo apresentado na Figura 4, foi realizada uma rotação de 45° em torno de um vetor unitário no eixo z ($v = [0, 0, 1]$). É importante mencionar que a rotação em torno do eixo z é equivalente a uma rotação em torno da origem no plano 2D, pois o eixo z funciona como o vetor normal perpendicular ao plano de visualização XY .

Como é possível comprovar pela figura abaixo, uma rotação pode traduzir-se numa translação.

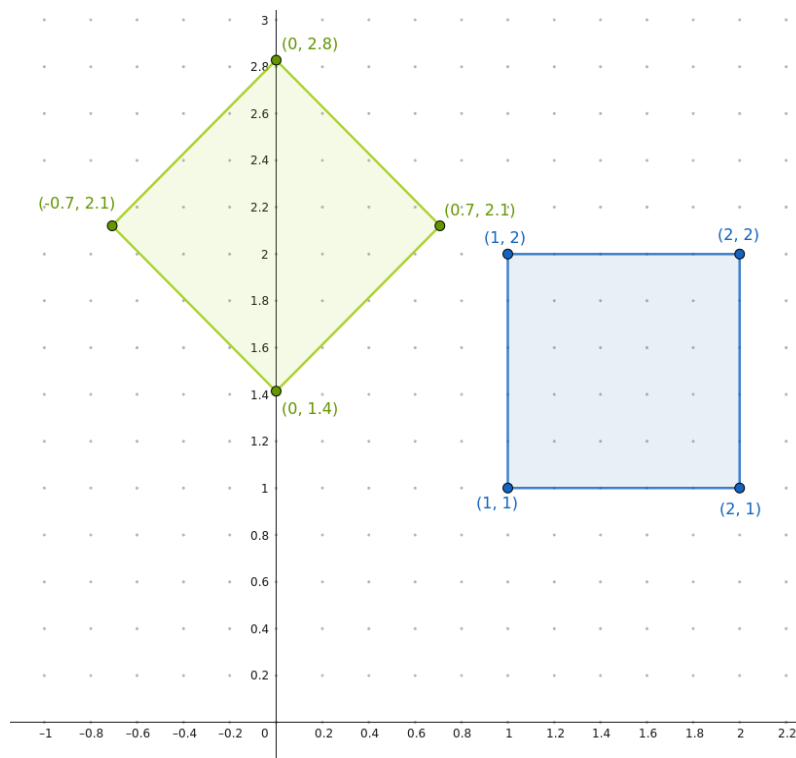


Figura 5 - Representação bidimensional da rotação de 45° em torno do eixo z ($v=[0,0,1]$).

1.2.3. Escala

A transformação geométrica de escala tem como objetivo redimensionar os modelos 3D presentes na cena. Esta operação altera as dimensões de um objeto multiplicando as coordenadas dos seus vértices por fatores definidos para os eixos x , y e z .

Matematicamente falando, utilizando coordenadas homogêneas, a escala é representada por uma matriz diagonal 4 x 4. A matriz de escala associada a um vetor de redimensionamento (s_x, s_y, s_z) é construída colocando os fatores na diagonal principal:

$$\mathbf{M} = \begin{pmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Figura 6 - Matriz de escala associada a um vetor (s_x, s_y, s_z).

Tal como mencionado anteriormente, no motor gráfico as matrizes são representadas no formato *column-major* e são pré-calculadas durante o *parsing*. No caso da escala, a classe *Group* possui a função *scale* que recebe como argumentos os valores do vetor (s_x, s_y, s_z) e substitui a matriz atual pela sua multiplicação com a matriz de escala desse vetor.

Ao aplicar um vetor de escala à matriz base, os objetos desenhados sofrem uma alteração nas suas proporções espaciais. Conforme ilustrado na Figura 6, este redimensionamento foi definido pelo vetor (0.5,0.75,0), modificando o tamanho dos elementos nos eixos correspondentes.

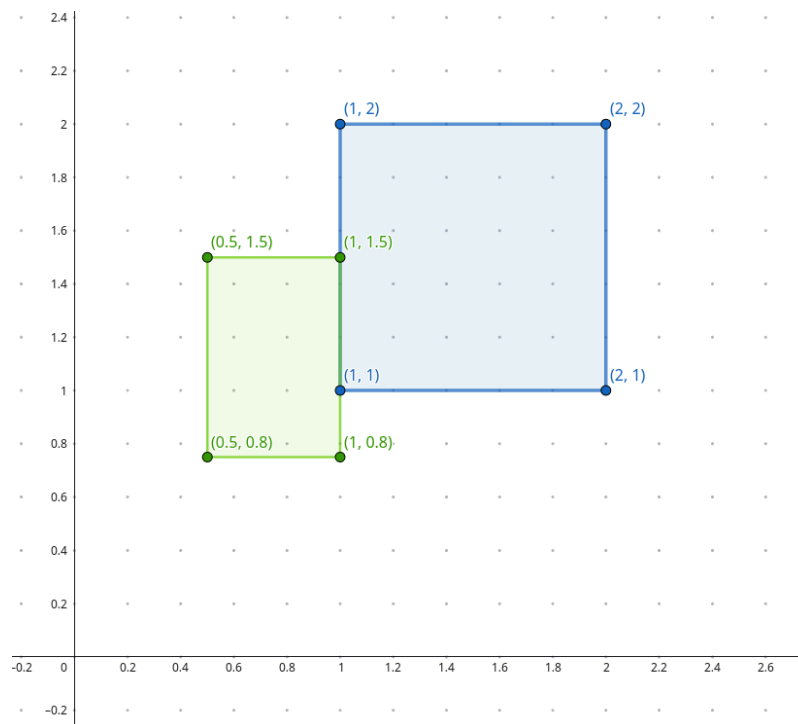


Figura 7 - Exemplo de uma escala segundo o vetor (0.5, 0.75, 0).

1.3. Desenho dos Grupos

Para desenhar os grupos respeitando a sua hierarquia, desenvolveu-se a função recursiva *drawGroup()*. Este algoritmo utiliza a pilha de matrizes do OpenGL da seguinte forma:

Primeiro, usamos *glPushMatrix()* para guardar o referencial do nó pai e *glMultMatrixf()* para aplicar as transformações ao grupo atual. É neste espaço que desenhamos os modelos do grupo e chamamos a função recursivamente para os seus subgrupos, garantindo que herdam estas mesmas

transformações. Por fim, invocamos *glPopMatrix()* para restaurar o estado original, evitando assim que as alterações deste grupo afetem indevidamente os restantes elementos da Scene.

2. Testes

Para testar as transformações implementadas nesta fase, utilizamos os ficheiros de teste disponibilizados pela equipa docente.

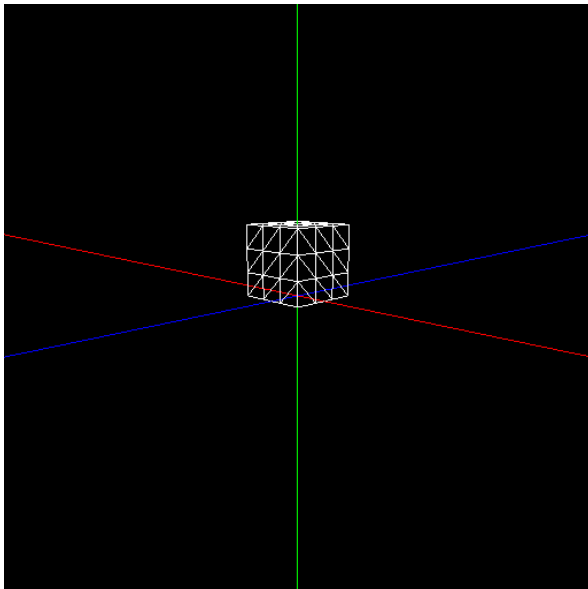


Figura 8 - Teste 1.

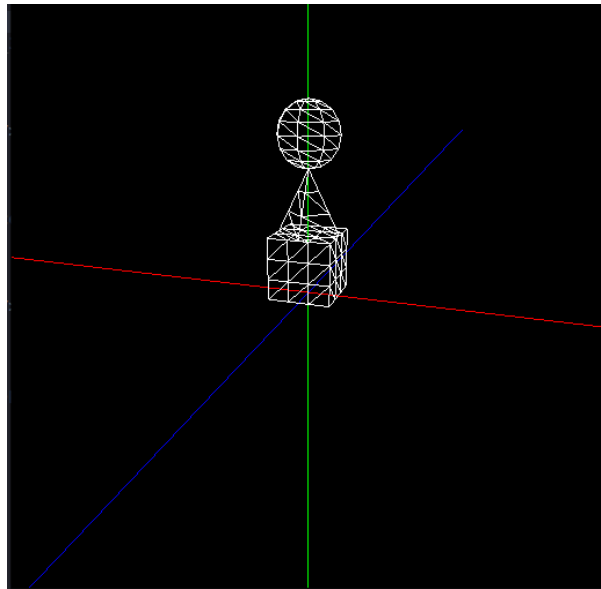


Figura 9 - Teste 2.

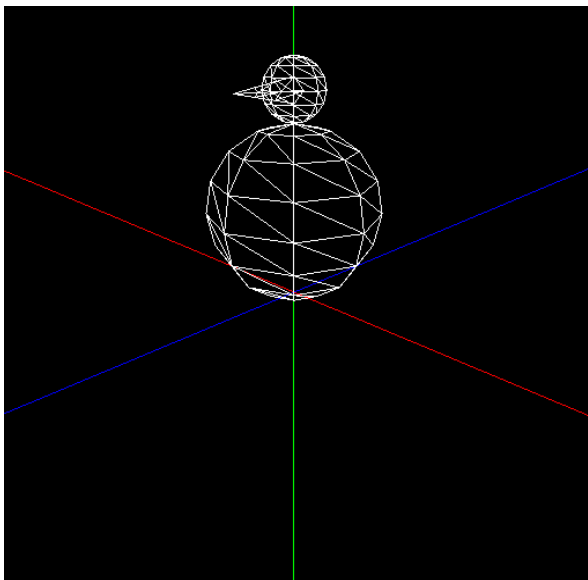


Figura 10 - Teste 3.

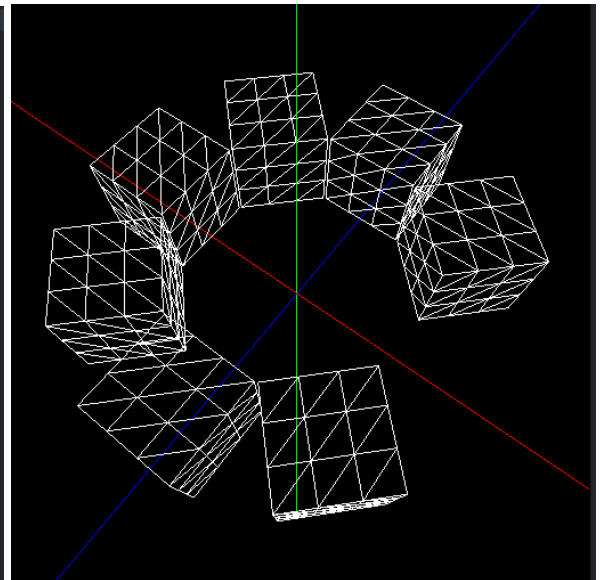


Figura 11 - Teste 4.

3. Sistema Solar

Para fins de visualização, foi desenvolvido um *script* que gera o ficheiro *XML* com o modelo estático do sistema solar conforme diferentes parâmetros. Este *script* permite a configuração das

escalas de diferentes componentes do sistema solar, incluindo a escala do sol, dos planetas e luas, da distância entre os planetas e luas e a variação máxima do ângulo de rotação da posição dos planetas em relação ao sol. A ordem dos argumentos recebidos pelo *script* é a seguinte:

```
./generate_solar_system.sh <sun_scale> <planet_scale> <dist_scale>  
<angle_variation>
```

Procurou-se preservar as proporções reais entre os vários corpos celestes, utilizando o raio da Terra como unidade de medida para os raios dos planetas e as distâncias deles ao Sol. Para as luas, utilizou-se a mesma abordagem, considerando a distância e o raio em relação ao planeta que orbitam, à exceção de algumas luas de Júpiter, Saturno, Urano e Neptuno. Devido à grande quantidade de luas que orbitam estes planetas, os valores de raio e distância de certas luas foram gerados aleatoriamente por uma questão de simplicidade de implementação.

No entanto, averiguou-se que a utilização de valores proporcionais à realidade para ambos os parâmetros de distância e raio resultava numa cena onde a maior parte dos modelos não eram renderizados. Assim, para garantir uma visualização adequada, as distâncias foram ajustadas de forma a que o argumento *dist_scale=1* correspondesse 0,1% da distância real.

Desta forma, o script gera um ficheiro *XML* onde cada grupo, à exceção do Sol, representa um planeta, que tem como transformações uma rotação aleatória para variar a sua posição orbital relativamente ao Sol, e uma translação e escala de acordo com os parâmetros definidos. Cada uma das luas é representada na forma de um subgrupo dentro do grupo do planeta que orbita. O modelo utilizado para representar tanto os planetas como as luas é a esfera, sendo também utilizado o torus para a representação dos anéis de Saturno.

Foi ainda adicionada uma cintura de asteroides entre Marte e Júpiter, onde a posição dos asteroides é gerada aleatoriamente dentro de uma área delimitada por uma distância média ao Sol, um tamanho radial e uma altura. O número de asteroides é constante e a escala dos asteroides é definida como 1% da escala dos planetas.

O ficheiro XML submetido nesta fase do trabalho (figura 10) foi gerado com os parâmetros:

```
./generate_solar_system.sh 0.9 4.5 0.3 -90
```

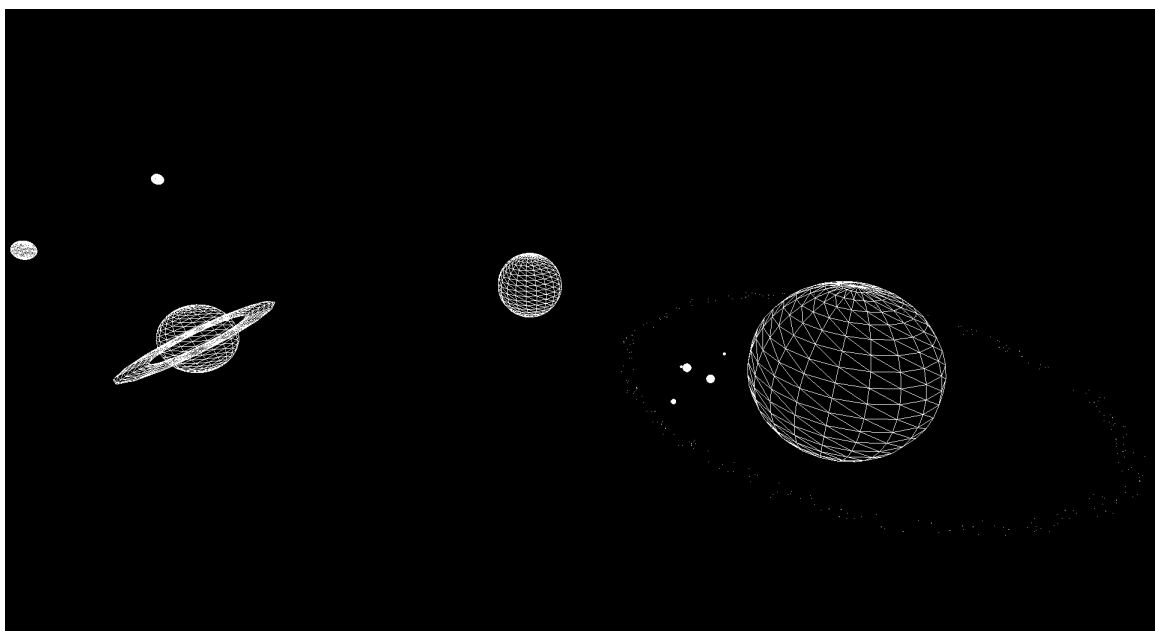


Figura 12 - Sistema solar gerado com parâmetros 0.9 4.5 0.3 -90.

4. Extras

4.1. Controlo da Câmara

De modo a facilitar a visualização da cena, foi implementado como um extra nesta fase o controlo da câmara através do teclado. O objetivo é permitir deslocar a câmara livremente na cena, movendo-se para trás, para a frente e lateralmente, alterando o *zoom* e rodando lateral em torno do ponto de focagem da câmara.

Consideremos os parâmetros da câmara utilizados na função *glutLookAt* especificados no *XML*,

- $P = (P_x, P_y, P_z)$ que representa a posição da câmara;
- $L = (L_x, L_y, L_z)$ que representa o ponto “Look at”, isto é, para onde a câmara olha;
- Vetor $\vec{up}$ que representa a orientação vertical da câmara;

e ainda os seguintes vetores,

- Vetor $\vec{d}$ que representa a direção do olhar da câmara, tal que $\vec{d} = L - P$;
- Vetor $\vec{r}$, perpendicular aos vetores $\vec{d}$ e $\vec{up}$, tal que $\vec{r} = \vec{d} \times \vec{up}$.

Relativamente ao movimento frontal e posterior da câmara, tem-se que a sua nova posição P' e a nova posição do ponto Look at L' são definidos por:

$$P' = P \pm kd$$

$$L' = L \pm kd$$

Onde k representa a magnitude do vetor de deslocamento. De forma semelhante, as posições obtidas com o movimento lateral da câmara são definidas por:

$$P' = P \pm kr$$

$$L' = L \pm kr$$

Já a alteração do *zoom* corresponde apenas a uma mudança da aproximação da posição da câmara relativamente ao ponto para onde ela olha. Sendo assim, apenas o ponto P é afetado, sendo que se considerarmos um fator de aproximação z , a sua nova posição é dada por:

$$P' = P \pm zd$$

A rotação implementada é feita em torno do eixo Y . Considerando α como sendo o ângulo de rotação, aplicando a transformação de rotação, tem-se que a nova posição de P é calculada da seguinte forma:

$$P' = (P'_x, P'_y, P'_z) = (L_x + (o_x \times \cos \theta - o_z \times \sin \alpha), P_y, L_z + ((o_x \times \sin \alpha + (o_z \times \cos \theta)))$$

onde o vetor

$$o = (o_x, o_z) = (P_x - L_x, P_z - L_z)$$

corresponde à distância relativa das projeções dos pontos P e L no plano XZ .

O efeito dado pela pressão de cada tecla encontra-se descrito abaixo.

w	→	move a câmara para a frente
s	→	move a câmara para trás
a	→	move a câmara para a esquerda
d	→	move a câmara para a direita
q	→	roda a câmara no sentido anti-horário em torno do eixo Y
e	→	roda a câmara no sentido horário em torno do eixo Y
x	→	aumenta o zoom
z	→	diminui o zoom

Conclusão

Com a evolução do motor gráfico, o *engine* permite a criação de estruturas de dados complexas, onde as transformações de translação, rotação e escala são acumuladas e herdadas através da pilha de matrizes do OpenGL. Além disso, agora é possível movimentar a câmara para uma melhor adaptação visual da cena.

A robustez do sistema foi demonstrada através dos ficheiros de testes dados e da modelação do Sistema Solar. Esta última cena prova a capacidade do engine em lidar com múltiplos níveis de profundidade (planetas e luas) e grandes volumes de objetos (cintura de asteroides), garantindo a integridade das proporções e posicionamentos espaciais.

Em suma, o trabalho realizado cumpre os objetivos definidos para a segunda fase do projeto.